The Chrome browser team recently released tools that allow Chrome to be run headless. In fact, as I write this, the maintainer for PhantomJS—which is a key dependency of Poltergeist—is stepping down because of headless Chrome. As of Capybara 2.15, there are default drivers for :selenium-chrome and :selenium-headless-chrome.

To use these drivers, you need a gem and a background application.

The gem is selenium-webdriver, and as of Rails 5.1 it is part of the Rails default gem set. The background application is ChromeDriver, which is basically the transfer protocol between Selenium and Chrome. On Mac OS, you can install it with homebrew using brew install chromedriver. Installation instructions for other platforms can be found online.[1]

We'll use :selenium-headless-chrome. Here's the configuration you can put into your existing Capybara configuration file:

js_integration/01/spec/support/system.rb

```
RSpec.configure do |config|
  config.before(:each, type: :system) do
    driven_by :rack_test
  end

  config.before(:each, type: :system, js: true) do
    driven_by :selenium_chrome_headless
  end
end

require "capybara-screenshot/rspec"
```

All you're doing here is registering :selenium-headless-chrome as a browser configuration that Selenium knows how to deal with, and then specifying that when Capybara needs a JavaScript-friendly driver in a system test, it should use headless Chrome.

Now you need to tell Capybara to use the JavaScript driver. In RSpec, you can set the JavaScript driver for a single test using metadata. The :js metadata that you've already added has Capybara switch to the JavaScript driver for the duration of the spec and then switch back at the end.

Prescription 24

Use Capybara's JavaScript integration-testing capabilities sparingly lest you be very, very annoyed. Test as much as possible within each layer, separate from the others.

1. https://sites.google.com/a/chromium.org/chromedriver/getting-started